
Evaluating Voting Design Vulnerabilities for Retroactive Funding

Jay Yu *

Stanford University
jyu01@stanford.edu

Austin Bennett

Stanford University
gaustinb@stanford.edu

Billy Gao

Stanford University
billygao@stanford.edu

Rebecca Joseph

Stanford University
rjoseph5@stanford.edu

Abstract

Retroactive Public Goods Funding (RetroPGF) rewards blockchain projects based on proven impact rather than future promises. This paper reviews voting mechanisms for Optimism’s RetroPGF, where “badgeholders” allocate rewards to valuable projects. We explore Optimism’s previous schemes for RetroPGF voting, including quadratic, mean, and median voting. We present a proof-based formal analysis for vulnerabilities in these voting schemes, empirically validate these vulnerabilities using voting simulations, and offer assessments and practical recommendations for future iterations of Optimism’s system based on our findings.

1 Introduction

1.1 Background and Motivation

Decentralized Autonomous Organizations (DAOs) have emerged as a novel form of organizational structure that leverages blockchain technology to enable transparent, automated decision-making processes [14]. Many different kinds of DAOs exist today in the blockchain industry, including protocol DAOs such as Uniswap, Optimism, and Arbitrum, collector DAOs such as PleasrDAO and NounsDAO, and investor DAOs. As of writing (November 2024), DAOs today manage over 22 billion USD in treasury value, and with major DAOs such as Optimism holding over 3 billion USD in treasury value [6].

Most DAOs today adopt some form of token-voting, where holders of a publicly-traded blockchain token, such as UNI, ARB, or OP are able to use their token holdings to cast votes on community proposals [7]. Token-based voting enables inclusive, community-driven participation, but practical challenges arise, including the risk of plutocratic capture by wealthy holders, low participation rates, and the need to balance efficiency with diverse interests [3]. Additionally, blockchain’s pseudonymity introduces vulnerabilities like Sybil attacks and vote manipulation.

These real-world stakes and relative ease of changing governance processes within DAOs (compared with traditional corporate or political settings) have allowed DAOs to become a fertile testing ground for testing governance mechanism innovation [13].

1.2 Optimism’s RetroPGF

Optimism’s Retroactive Public Goods Funding (RetroPGF) represents an innovative program to sustaining public goods development in the blockchain ecosystem. Unlike traditional grant programs

*Funded in part through a research grant by the Optimism Foundation

that fund projects based on prospective promises, RetroPGF rewards projects retroactively based on their demonstrated impact. This mechanism aligns incentives by ensuring that funding flows to projects that have already proven their value to the ecosystem [8].

The RetroPGF system operates through rounds of funding, where qualified voters (badgholders) in Optimism’s Citizen House evaluate and allocate rewards to projects that have contributed value to the Optimism ecosystem. The process begins with project teams submitting evidence of their contributions, followed by the careful selection of community members with demonstrated expertise as badgholders. These badgholders then engage in a thorough evaluation period, reviewing and assessing project contributions before their votes are aggregated to determine the final funding distribution.

This system presents unique challenges for voting mechanism design. The need to balance the expertise of specialized voters with broad community representation creates an inherent tension in the selection process. The system must prevent collusion while allowing voters to correctly express their project preferences. To address this, Optimism has experimented with different mechanisms in each of the four rounds of prior RetroPGF funding [8].

- Round 1 used a method of **Quadratic Voting**
- Round 2 used a method of **Mean Voting**
- Round 3 and Round 4 used **Variants of Median Voting**

We will be exploring the voting designs and potential vulnerabilities of these various voting mechanisms in the following sections of this paper.

1.3 Contributions

This paper makes several contributions to the the analysis and development of Retroactive funding processes:

1. We situate Optimism’s RetroPGF within the broader context of social choice theory and offer a formal model for RetroPGF
2. We provide proofs and analysis of vulnerabilities within various RetroPGF processes
3. We simulate RetroPGF voting processes to determine the practical impact of these theoretical vulnerabilities
4. We provide specific recommendations for the implementation of voting mechanisms in future RetroPGF rounds and provide an open-source dashboard for practitioners to replicate and extend our findings.

The remainder of this paper is organized as follows: Section 2 provides an overarching literature review to situate RetroPGF within a broader context of social choice theory. Section 3 outlines our methodology and introduces a formal model for our proof-based evaluation and simulation framework. Section 4 details our proof-based analysis of vulnerabilities in the target voting mechanisms, and Section 5 assesses their practical impact in a simulation context. Finally, we present our findings in Section 6 and provide recommendations for further iterations of RetroPGF.

2 Literature Review: Social Choice Theory and DAOs

2.1 Goals of Social Choice Theory

Social choice theory examines how individual preferences can be aggregated into collective decisions while balancing fairness, efficiency, and strategic incentives [16]. Its modern formulation traces back to Condorcet’s work, which revealed that majority preferences could be cyclical: even with rational individual preferences, collective choices might yield intransitive outcomes where group preferences cycle between alternatives [5]. This observation, later known as Condorcet’s Paradox, highlighted fundamental tensions in preference aggregation that continue to influence modern mechanism design, particularly the challenge of achieving *strategyproof-ness*, where truthful participation is optimal regardless of others’ actions.

Building on Condorcet’s early work, two landmark results frame the modern study of social choice theory. Arrow’s Impossibility Theorem [1] proves no ranked voting system can simultaneously satisfy non-dictatorship, Pareto efficiency, and independence of irrelevant alternatives when there are three or more options. Gibbard-Satterthwaite’s theorem [10] extends this by showing all non-dictatorial voting schemes are susceptible to tactical voting unless restricted to binary choices. Together, these results establish fundamental limits on mechanism design: we cannot simultaneously achieve basic democratic principles and strategy-proofness except in highly restricted settings, shaping the problem space for modern DAO mechanism design.

2.2 Theoretical Foundations for RetroPGF Mechanisms

Optimism’s RetroPGF experiments operationalize social choice principles through three distinct voting mechanisms, quadratic voting in Round 1, mean voting in Round 2, and variants of median voting in Rounds 3 and 4. Here, we examine the core theoretical justifications for each voting mechanism.

Quadratic voting, used in RetroPGF Round 1, was developed by economists E. Glen Weyl and Steven P. Lalley [15], attempting to build on the earlier Vickrey-Clarke-Groves (VCG) mechanism to allow voters to express preference intensity [17]. In a token-based quadratic voting system, a voter’s voting power is determined by the square root of the token holdings, allowing participants to express preference intensity while mitigating risk of power concentration among wealthy. Quadratic voting achieves allocative efficiency by imposing convex marginal costs on voting power, where the square root scaling ensures each additional unit of influence becomes exponentially more expensive. This convexity aligns individual expenditure with marginal utility, inducing voters to distribute tokens proportionally to their true valuations, thereby approximating welfare-maximizing equilibria observed in competitive markets [17].

Mean voting, utilized in Round 2, has a less formally established framework than quadratic voting and median voting, but is conceptually very simple and builds on classic aggregation theory in statistics and economics[9]. Mean voting involves allocating funding according to the average of all votes received for each project, achieving representational accuracy by incorporating all votes equally into the final outcome, allowing full expression of preferences across the spectrum. However, this makes it vulnerable to manipulation through extreme values, just like linear voting. The mechanism optimizes for aggregate preference revelation, theoretically maximizing utilitarian welfare under assumptions of honest voting and normal distribution of preferences [19].

Median voting, adapted in Rounds 3 and 4 of Optimism’s RetroPGF program, builds on Duncan Black’s median voter theorem [4], in which Black argues that the median allocation is robust against outlier values. The mechanism induces convergence toward centrist outcomes, reflecting the theoretical prediction that competing interests will gravitate toward the median voter’s preference. However, its ordinal (vs cardinal) nature discards preference intensity data, creating openings for threshold-based attacks absent in interval-scale systems, such as phantom vote style attacks that we will describe in Section 4.

2.3 Cross-Mechanism Comparison with DAOs

Traditionally, each of the above mechanisms have been analyzed and theorized in isolation, often in the context of existing political and economic systems. However, the rapid ascent of DAOs provide a new paradigm for studying multiple voting mechanism innovations in conjunction with one another in a setting with real-world stakes [13]. In particular, Optimism’s sequential adoption of multiple mechanisms, from quadratic (Round 1), mean (Round 2), and median (Rounds 3-4) voting provide a valuable real-world testbed for comparative vulnerability analysis under shared model constraints, which are described in Section 3.

In our work, we present a cross-mechanism comparison through a dual-approach, creating formalized proofs of different vulnerabilities and attack strategies on each of the target mechanisms, and a simulation framework that seeks to quantify and compare the resulting impacts of each minimum viable attack strategy on the various voting mechanisms. Our work thus provides a common baseline and lens of analysis to assess the comparative strengths and weaknesses of these various mechanisms from social choice theory, while also giving recommendations to DAO practitioners on how to design a more robust voting system for RetroPGF.

3 Methodology

As mentioned in Section 1, Optimism has previously conducted RetroPGF using three different mechanisms across their four rounds: quadratic voting, mean voting, and median voting. Our approach will focus on both a proof-based theoretical analysis of these voting mechanisms to expose potential vulnerabilities in each of these models in Section 4. We then empirically verify and assess the impact of these vulnerabilities in simulated voting rounds in Section 5 and discuss our results in Section 6. In this section, we will outline the formal model of Optimism’s RetroPGF setup that we will analyze, both in our formal analysis and our simulations.

3.1 Optimism RetroPGF Governance Assumptions

In their RetroPGF program, Optimism incentivizes development by providing retroactive funding to projects based on their success, which is determined by a group of pre-selected badge holders. To model this process, in this paper, we apply the following assumption model:

1. There is a fixed number of voters in our system.
2. A voter cannot split their funds into multiple voting wallets.
3. There is a fixed number of projects where voters can allocate funds.
4. Voters all have a set number of tokens to allocate.
5. Rational voters will maximize their utility – a function of their allocations to projects.

3.2 Model Specification

Consider a single-shot Optimism RetroPGF round with $n \in \mathbb{Z}^+$ voters and a total funding allocation of $T \in \mathbb{R}^+$ tokens to be evenly divided among them. This yields a voting power of $\frac{T}{n}$ for each voter. We assume that there are no costs associated with casting a vote. Thus, we define the following model to represent a funding round:

- The number of voters within a retroactive funding round: $N \in \mathbb{Z}^+$
- The number of projects available for funding: $P \in \mathbb{Z}^+$
- The number of tokens to be distributed: $T \in \mathbb{R}^+$
- The Allocation Matrix, $A \in \mathbb{R}^{N \times P}$, where A_{ij} is equal to voter i ’s allocation to project j .

Subject to the Following Model Variable Constraints

1. All voters have a maximum of $\frac{T}{N}$ votes: $\sum_j^P A_{ij} \leq \frac{T}{N}, \forall i \in [1, \dots, N]$
2. Vote amounts are non-negative: $A \geq 0$

3.3 A Simplified Model of Aggregate Voting Mechanisms

3.3.1 Mean Voting

Given the allocation matrix $A \in \mathbb{R}^{N \times P}$, where A_{ij} represents the allocation of tokens from voter i to project j , the total allocation for each project j is calculated as:

$$T_j = \sum_{i=1}^N A_{ij}$$

The allocation is given by:

$$P_j = \frac{T_j}{\sum_{j=1}^P T_j}.$$

$$\text{Allocation} = T \cdot P_j$$

3.3.2 Median Voting

Given the allocation matrix $A \in \mathbb{R}^{N \times P}$, where A_{ij} represents the allocation of tokens from voter i to project j , the median allocation for each project j is calculated as:

$$M_j = \text{median}\{A_{ij} \mid A_{ij}, i = 1, \dots, N\},$$

The allocation is given by:

$$P_j = \frac{M_j}{\sum_{j=1}^P M_j},$$

$$\text{Allocation} = T \cdot P_j$$

3.3.3 Quadratic Voting

Given the allocation matrix $A \in \mathbb{R}^{N \times P}$, where A_{ij} represents the allocation of tokens from voter i to project j , the quadratic voting allocation for each project j is calculated as:

$$Q_j = \sum_{i=1}^N \sqrt{A_{ij}},$$

The allocation is given by:

$$P_j = \frac{Q_j}{\sum_{j=1}^P Q_j}.$$

$$\text{Allocation} = T \cdot P_j$$

4 Voting Mechanism Analysis

4.1 Quadratic Voting

First proposed by Lalley and Weyl [15], a quadratic voting mechanism reweighs votes by valuing them as the square roots of the allocation. This has many advantages with respect to economic optimality and decentralization; however, in many blockchain systems it is difficult to implement due to the lack of identity verification and the possibility of Sybil attacks, where an adversary could split their tokens among an arbitrarily large number of wallets and gain an outsize influence on the network.

In the context of Optimism's RetroPGF program, however, this is not a concern as voters cannot split their token votes among some large number of wallets. Scenarios resembling Sybil attacks could still persist should multiple voters decide to collude. Specifically, two voters with strong preferences for one project who collude can artificially increase their voting power by a factor $\sqrt{2}$. To see this, consider the optimization problem.

Remark 1 *The Quadratic Collusion Attack – For two colluding voters, who care solely about the success of projects p and q , respectively, optimal collusion for both parties under Quadratic Voting allows them to each artificially increase their voting power by a factor of $\sqrt{2}$.*

Consider two colluding voters have a total of T tokens who can contribute (p_1, q_1) and (p_2, q_2) to projects p and q , respectively, yielding total contributions of $P = p_1 + p_2$ and $Q = q_1 + q_2$. Additionally, assume that these voters only care about the number of funds allocated to projects p and q , respectively. Holding all other allocations by voters constant, this yields the simplified utility functions:

- Voter 1 Utility: $u_1(p_1, q_1) = \sqrt{p_1} + \sqrt{q_1}$
- Voter 2 Utility: $u_2(p_2, q_2) = \sqrt{p_2} + \sqrt{q_2}$

To find optimal allocations for both colluding parties, we know that neither will voter will allow the other project to amass more of their collective resources, as this would be suboptimal since voting power increases with votes allocated. Thus, we know that $P = Q = T$.

Now, the task is to find the optimal allocation among p_1 , p_2 , q_1 , and q_2 . Our constraints only restrict the relationship between p_1 and p_2 as well as q_1 and q_2 , so we can treat these as separate optimization problems and solve for the optimal value for our decision variables.

Starting with p_1 and p_2 , we have the following optimization problem:

$$\max_{p_1, p_2} \sqrt{p_1} + \sqrt{p_2}$$

Subject to:

1. $p_1 + p_2 = P$
2. $p_1 \geq 0$
3. $p_2 \geq 0$

The second and third constraints are satisfied by our model definition. Additionally, by the first constraint, we can rewrite $p_2 = P - p_1$. Substituting this into our optimization problem, we obtain a single-variable objective function to maximize. Taking the first order conditions, we can solve for the optimal value as p_1 as follows:

$$\begin{aligned} \max_{p_1} \sqrt{p_1} + \sqrt{P - p_1} &\implies \frac{\partial}{\partial p_1} (\sqrt{p_1} + \sqrt{P - p_1}) = 0 \\ &\implies \left(\frac{1}{2\sqrt{p_1}} - \frac{1}{2\sqrt{P - p_1}} \right) = 0 \\ &\implies \frac{1}{2\sqrt{p_1}} = \frac{1}{2\sqrt{P - p_1}} \\ &\implies \sqrt{p_1} = \sqrt{P - p_1} \\ &\implies p_1 = P - p_1 \\ &\implies 2p_1 = P \\ &\implies p_1 = \frac{P}{2} \end{aligned}$$

From here, we see that the optimal allocation for $(p_1^*, p_2^*) = (\frac{P}{2}, \frac{P}{2})$, yielding the optimal utility:

$$u_1^*(p_1^*, p_2^*) = 2\sqrt{\frac{P}{2}} = \sqrt{2} \cdot \sqrt{P} = \sqrt{2} \cdot \sqrt{T}$$

Without loss of generality, the same argument can be applied in solving for the optimal $(q_1^*, q_2^*) = (\frac{Q}{2}, \frac{Q}{2})$, yielding the optimal utility:

$$u_2^*(q_1^*, q_2^*) = 2\sqrt{\frac{Q}{2}} = \sqrt{2} \cdot \sqrt{Q} = \sqrt{2} \cdot \sqrt{T}$$

In both cases, we see that the colluding parties, each of whom in theory have a voting power of $\sqrt{T}$, can multiply their voting power by a factor of $\sqrt{2}$ by agreeing to use half of their tokens to vote for the other project. $\square$

4.2 Mean Voting

Mean voting is a mechanism where we allocate funding according to the mean of the vote amounts for a project. Here, we show the importance of including all wallets under the aggregation process, including those with a vote of zero tokens.

Remark 2 *When all vote allocations are included, then Mean Voting is a Linear Voting Mechanism*

When all votes are included, notice that for a project, j in our model, $j \in [1, \dots, P]$, we can represent its allocation total as:

$$\text{Project } j\text{'s Mean Votes Received} = \frac{1}{N} \sum_{i=1}^N A_{ij}$$

Since N is a constant, we see that mean allocation has a linear relationship to the number of tokens allocated to the project. This is proportionally the same as Linear Voting; and, therefore, offers no strategic advantage in resources allocation. It may, however, yield some advantages with respect to ease of determining token allocation. For example, mean voting could allow groups to make decisions about the allocation proportion from a larger pool in an easy-to-view way for voters.

Remark 3 *The Mean Phantom Vote Attack – Where n is the number of non-zero wallets, Mean Voting without zero-value allocations included can lead k adversaries to artificially reduce the vote total by up to $\frac{n}{n+k}$, where $k \leq N - n, k \in \mathbb{Z}^+$.*

To see this, choose some arbitrarily project j with n non-zero allocations and the allocation vector $a_j = (a_{1j}, \dots, a_{nj})$. Notice that this is not the same as the allocation matrix, as this is simply the non-zero subset of the column associated with allocations for a project, j . Thus, the mean voting power prior to an attack simply takes the form:

$$\text{Project } j\text{'s Mean Voting Power before Attack} = \frac{1}{n} \sum_{i=1}^n a_{ij}$$

Now, say some, k , adversaries don't like this project and want to decrease its mean vote total. This can be for a variety of reasons, though most commonly to reserve funds for another project.

The adversaries can each choose some arbitrarily small $\epsilon_k > 0$ for them to commit to a project, j . This yields, with a slight abuse of notation, the following mean voting power after an attack:

$$\text{Project } j\text{'s Mean Voting Power after Attack} = \lim_{\epsilon_i \rightarrow 0^+} \frac{1}{n+k} \left(\sum_{i=1}^n a_{ij} + \sum_{i=1}^k \epsilon_i \right) = \frac{1}{n+k} \sum_{i=1}^n a_{ij}$$

From this, we see that the relative mean voting power of a project after the attack compared to before the attack can be simplified as follows:

$$\frac{\text{Project } j\text{'s Mean Voting Power after Attack}}{\text{Project } j\text{'s Mean Voting Power before Attack}} = \frac{\frac{1}{n+k} \sum_{i=1}^n a_{ij}}{\frac{1}{n} \sum_{i=1}^n a_{ij}} = \frac{n}{n+k} \square$$

This problem arises from the number of voters which we count in the mean vote total. Since an adversary can contribute effectively nothing, they can decrease the average number of vote allocation. If we count all vote allocations, even with zero value, then we have the constant number of participants, N , yielding a scaling term of $\frac{1}{N}$ for all vote allocation totals. This yields the linear system described in Remark 1.

4.3 Median Voting

Median voting is a mechanism where we allocate funding according to the median of the vote counts for a project. Here, we show how the phantom vote attack may be more dangerous for median voting, as we cannot rely on including all zero votes as we did with mean voting.

Remark 4 *The Median Phantom Vote Attack – Where n is the number of non-zero wallets, let $a_j^{(i)}$ be the i th order statistic for $i \in [1, \dots, n]$. If we have a median allocation a_m , where m is the largest index such that $a_m \leq \text{median}(a)$, then Median Voting without zero-value allocations included can lead k adversaries to artificially decrease the voting power to where it is bounded above by $a_j^{(m - \lceil \frac{k}{2} \rceil + 1)}$.*

To see this, invoke a similar strategy as described in the "mean phantom vote" section. More specifically, each adversary, k , can each choose some corresponding arbitrarily small $\epsilon_k > 0$ for them to commit to a project, j , such that $\epsilon_k < a_1$ prior to the attack.

Because the calculation of median depends on the number of elements, we now consider four cases:

Case: n is even, k is even

In this case, we see $n + k$ is even. Prior to the attack, the median value is equal to $\frac{a_j^{(m)} + a_j^{(m+1)}}{2}$. Because we now have k elements below our median, by the definition of median we see that our median value shifts down $\frac{k}{2}$ order statistics and is now $\frac{a_j^{(m - \frac{k}{2})} + a_j^{(m - \frac{k}{2} + 1)}}{2}$, as we still have an even number of allocations and need to take the average of the allocations on either side.

This is bounded above by $a_j^{(m - \lceil \frac{k}{2} \rceil + 1)}$, as we have:

$$a_j^{(m - \frac{k}{2})} \leq a_j^{(m - \lceil \frac{k}{2} \rceil + 1)} \implies \frac{a_j^{(m - \frac{k}{2})}}{2} \leq \frac{a_j^{(m - \lceil \frac{k}{2} \rceil + 1)}}{2} \implies \frac{a_j^{(m - \frac{k}{2})} + a_j^{(m - \lceil \frac{k}{2} \rceil + 1)}}{2} \leq a_j^{(m - \lceil \frac{k}{2} \rceil + 1)}$$

Case: n is even, k is odd

In this case, we see $n + k$ is odd. Prior to the attack, the median value is equal to $\frac{a_j^{(m)} + a_j^{(m+1)}}{2}$. Because we now have k elements below our median, by the definition of median we see that our median value shifts down $\frac{k-1}{2}$ order statistics from a_j^m and is now $a_j^{(m - \frac{k-1}{2})}$.

This is bounded above by $a_j^{(m - \lceil \frac{k}{2} \rceil + 1)}$.

Case: n is odd, k is even

In this case, we see $n + k$ is odd. Because we now have k elements below our median, by the definition of median we see that our median value, $a_j^{(m)}$, shifts down $\frac{k}{2}$ order statistics and is now $a_j^{(m - \frac{k}{2})}$.

This is bounded above by $a_j^{(m - \lceil \frac{k}{2} \rceil + 1)}$.

Case: n is odd, k is odd

In this case, we see $n + k$ is even. Because we now have k elements below our median, by the definition of median we see that our median value, $a_j^{(m)}$, shifts down $\frac{k-1}{2}$ order statistics and is now

$$\frac{a_j^{(m - \frac{k-1}{2})} + a_j^{(m - \frac{k-1}{2} + 1)}}{2}.$$

This is bounded above by $a_j^{(m - \lceil \frac{k}{2} \rceil + 1)}$.

Thus, we see this bound holds in all cases so the median phantom vote attack is proven. While perhaps more difficult to quantify, the phantom vote attack may be more dangerous in this setting than mean voting. This is because if a large gap between two allocations is identified, it could be targeted by a sufficient number of adversaries to exploit, causing a much larger jump downward than we see with mean voting. For this reason, we may wish to implement alternative measures, such as a minimum allocation threshold in this case.

5 Voting Mechanism Simulations

5.1 Experimental Design

This study uses a simulation-based framework to evaluate the performance and robustness of various voting mechanisms in Optimism’s RetroPGF process ². Our simulation runs 10,000 iterations with 133 voters and 374 projects, parameters chosen to match the scale of real-world RetroPGF rounds [2].

For each voting mechanism (Quadratic, Mean, and Median), our experimental evaluation focuses on two primary attack vectors:

1. **Voter Attacks:** Scenarios where groups of voters coordinate to manipulate outcomes by strategically concentrating their voting power on specific projects.
2. **Project Attacks:** Scenarios where multiple projects collude by encouraging their supporters to split votes among the colluding group, effectively amplifying their collective allocation.

In addition to these two attack scenarios, we also model a baseline allocation for each target voting mechanism. We measure the voting mechanisms resilience through a pairwise comparison between the baseline allocation and post-attack allocations, defined in Section 5.4, to compute a Pairwise Manipulation Score that we compare across different attack scenarios.

5.2 Preference Construction

Our simulation results are based on a voter preference matrix, denoted M , representing the relative preferences of the voters distributed throughout all the projects, roughly corresponding to the Allocation Matrix A in Section 3. Each element of the matrix, $M_{v,p}$ represents the preference of voter v for project p , normalized such that the preferences for each voter sum to 1. For our simulation, we construct M voter by voter such that:

$$M_{v,p} = \frac{X_{v,p}}{\sum_p X_{v,p}} \quad \forall v \quad (1)$$

where $X_{v,p}$ is sampled from a Pareto distribution:

$$X_{v,p} \sim \text{Pareto}(\alpha = 2.5) \quad \forall v, p \quad (2)$$

This construction ensures:

$$M_{v,p} \in [0, 1] \quad \forall v, p \quad (3)$$

$$\sum_p M_{v,p} = 1 \quad \forall v \quad (4)$$

We generate M using a Pareto distribution to better model empirical observations that each individual voter may only have time to thoroughly evaluate a small subset of total projects, as well as herding and other naturally occurring centralizing vectors for preference distribution [18]. The shape parameter $\alpha = 2.5$ models moderate preference concentration while maintaining finite variance in the distribution, allowing for moderately heavy tails while also accounting for preference concentration to better match empirical observations for RetroPGF rounds. Our simulation framework also supports

²The complete codebase used for the simulation can be found at <https://github.com/Stanford-Blockchain-Club/opt-voting>

alternative distributions of $X_{v,p}$ such as sampling from a uniform or Gaussian distribution, or directly importing voter preference data.

In addition to matrix M , our simulation framework also accounts for a weight vector W , where each $w_v \in W$ accounts for voter v 's vote weights. As per our Optimism RPGF model specification in Section 3, in our experiment $\forall v (w_v = 1)$. Then, voter weights are normalized such that $\sum_v w_v = c$, where c is a normalization constant chosen large enough for floating point numerical stability (eg. 1000).

5.3 Voting Mechanisms

As outlined in Section 5.1, we examine how three different voting mechanisms (Quadratic, Mean, and Median) that are used in prior RetroPGF rounds behave under Voter Attack and Project Attack scenarios. These two attack scenarios represent two different sets of incentives for attacking, one from a voter perspective, wanting their chosen projects to win, and the other from a project perspective. We also add a baseline mechanism that simply sums up voter preferences to act as a control. Table 1 provides an overview of the different strategies in each of the voting mechanisms and attack scenarios.

Table 1: Voting Mechanisms and Attack Scenarios

Mechanism	Baseline	Voter Attack	Project Attack
Baseline	$R_p = \sum_v M_{v,p} \cdot w_v$	N/A	N/A
Quadratic	$R_p = \sum_v \sqrt{M_{v,p} \cdot w_v}$	Colluding pairs split $\sqrt{0.5w_v}$ between top preferences	Projects pool supporters to split votes among colluding group
Mean	$R_p = \frac{1}{ V } \sum_v M_{v,p} \cdot w_v$	Attacker assigns ε to all except max preference	Project supporters minimize votes to other projects (ε)
Median	$R_p = \text{median}_v(M_{v,p} \cdot w_v)$	Similar to mean attack with ε strategy	Similar to mean attack with coordinated ε assignments

Note: $M_{v,p}$ represents normalized preferences where $\sum_p M_{v,p} = 1$ for each voter v . w_v is voter v 's voting power. ε represents a small positive constant (0.01).

5.3.1 Baseline Mechanism

The baseline mechanism directly aggregates normalized voter preferences, providing a benchmark for comparing other mechanisms:

$$R_p = \sum_v M_{v,p} \cdot w_v \quad (5)$$

where $M_{v,p}$ represents normalized preferences such that $\sum_p M_{v,p} = 1$ for each voter, serving just as a reference point for measuring manipulation in other mechanisms.

5.3.2 Quadratic Voting

Quadratic voting applies a square root transformation to voting power to reduce the impact of vote concentration:

$$R_p = \sum_v \sqrt{M_{v,p} \cdot w_v} \quad (6)$$

Under the voter attack scenario, a pair of colluding voters ($v_1, v_2 \in C$) coordinate by splitting their voting power between their highest-ranked preferences:

$$R_p = \begin{cases} \sqrt{0.5w_v} & \text{if } p \in \text{top}_2(M_{v,p}), v \in C \\ \sqrt{M_{v,p} \cdot w_v} & \text{otherwise} \end{cases} \quad (7)$$

Project attacks occur when an attack project (P_c) coordinate their supporters (V_p) to split votes among the colluding group:

$$R_p = \begin{cases} \sqrt{0.5w_v} & \text{if } p \in P_c, v \in V_p \\ \sqrt{M_{v,p} \cdot w_v} & \text{otherwise} \end{cases} \quad (8)$$

5.3.3 Mean Voting

Mean voting averages normalized allocations across all voters:

$$R_p = \frac{1}{|V|} \sum_v M_{v,p} \cdot w_v \quad (9)$$

This mechanism adapts phantom voting techniques from our theoretical framework for a fixed voter set, as required in our model in Section 3. Under voter attacks, an attacker allocates minimum votes (ε) to all projects except their target to manipulate the mean:

$$R_p = \frac{1}{|V|} \sum_v \begin{cases} w_v - (|P| - 1)\varepsilon & \text{if } p = p_v^* \\ \varepsilon & \text{otherwise} \end{cases} \quad (10)$$

Project attacks follow a similar pattern but coordinate across supporter groups:

$$R_p = \frac{1}{|V|} \sum_v \begin{cases} w_v - (|P| - 1)\varepsilon & \text{if } p \in P_c, v \in V_p \\ \varepsilon & \text{otherwise} \end{cases} \quad (11)$$

5.3.4 Median Voting

As in our theoretical framework, median voting follows similar phantom attack patterns to mean voting but uses the median to reduce sensitivity to extreme allocations. We again use the ε pattern to model these here. For voter attacks:

$$R_p = \text{median}_v \begin{cases} w_v - (|P| - 1)\varepsilon & \text{if } p = p_v^* \\ \varepsilon & \text{otherwise} \end{cases} \quad (12)$$

Project attacks similarly coordinate across supporter groups:

$$R_p = \text{median}_v \begin{cases} w_v - (|P| - 1)\varepsilon & \text{if } p \in P_c, v \in V_p \\ \varepsilon & \text{otherwise} \end{cases} \quad (13)$$

For each mechanism, we implement minimum viable attacks — the simplest attack patterns that demonstrate the mechanism vulnerabilities that consist of having a minimum amount of voters or projects participating in an attack type.

5.4 Evaluation Metric

To quantify the resilience of voting mechanisms against manipulation, we introduce a Pairwise Manipulation Score (PMS), a modification of Mean Squared Error (MSE) adapted specifically for voting distribution analysis. While MSE measures absolute differences between distributions, PMS is designed to capture relative changes in voting power distribution.

For vote distributions v_1, v_2 over n projects, we first normalize to percentage allocations:

$$p_i^k = \frac{v_k[i]}{\sum_{j=1}^n v_k[j]} \times 100 \quad (14)$$

where p_i^k represents the percentage of votes allocated to project i in distribution k . The Pairwise Manipulation Score is then defined as:

$$\text{PMS}(v_1, v_2) = \frac{\sum_{i=1}^n (p_i^1 - p_i^2)^2}{\sum_{i=1}^n (p_i^1)^2} \times 100 \quad (15)$$

Unlike traditional MSE which takes the form $\frac{1}{n} \sum_{i=1}^n (y_{\text{true}} - y_{\text{pred}})^2$, our metric creates two key modifications, with percentage normalization to ensure scale invariance across different voting mechanisms and total vote counts, and division over the sum of squared baseline percentages to measure relative rather than absolute changes. For each voting mechanism M and attack scenario A , we can thus compute:

$$\text{Manipulation}(M, A) = \text{PMS}(\text{BaselineResults}(M), \text{AttackResults}(M, A)) \quad (16)$$

A PMS of 0 indicates identical distributions, while higher scores represent increasing levels of vote distribution distortion, quantitatively comparing the resilience of different voting mechanisms in the different attack scenarios.

6 Results

6.1 Theoretical Evaluation

Drawing from the proofs of our voting model in Section 4, we have a few observations:

Observation 1: Median Voting is More Vulnerable to Large-Scale Attacks than Mean Voting

While both measures attempt to find some notion of a "middle" allocation, notice that the upper bounds of our Mean Phantom Vote Attack and Median Phantom Vote Attack are quite different. The former relies on a normalized sum, while the latter relies on an ordering of the allocations.

For this reason, each allocation will have an equal impact on the value of Mean Voting, while only the center allocations impact the value of a Median Voting, notwithstanding the actual order of the allocations themselves. Thus, an attacker could cause much massive swings within a median value, especially if there are large gaps in value between the allocations.

To get an intuition for why this difference matters, consider the following set of allocations for a project: $[0, 0, 0, 0, 100, 100, 100, 100]$. Currently, both mechanisms yield a value of 50.

If an adversary contributes 0 to both projects, we now have $[0, 0, 0, 0, 100, 100, 100, 100]$, yielding:

$$\text{Mean Voting} = \frac{1}{9} (5 \cdot 0 + 4 \cdot 100) = 44.\bar{4}$$

$$\text{Median Voting} = 5\text{th Ranked Value} = 0$$

This is an extreme example, as median is unlikely to be swayed this much by a single vote; but in as we've shown in Section 4.2, many adversaries can attack to bring down the median. Additionally, we can't simply count zero-valued votes as we did in Mean Voting, as this would also bring down the median to a point where it could wash out all real allocations.

There have been some clever workarounds proposed to address other issues in Median Voting, such as the use of Capped Median Rule and Moving Phantoms Rules that could provide some sort of stopgap against this attack due to making this drop off less severe. Additionally, raising the threshold of a minimum allocation and other floors and ceilings within mechanism design could make this sort of attack prohibitively expensive. We discuss these additional considerations in Section 7.

Observation 2: Quadratic Voting Exploitation Occurs with Repeated Voters

While the prospect of increasing vote power by a factor $\sqrt{2}$ is quite substantial, this only works so long as two delegates have an incentive to collude.

If the same delegates are consistently used for every vote, over time the benefits of collusion could outweigh the benefits of betraying the effort. Its possible that this repeated collusion never reaches a Nash Equilibrium; but if the delegates are frequently switched, it becomes substantially less likely as the delegates would almost certainly value future collusion less.

At a high-level, if a colluder is going to contribute half of their tokens to the project that someone wants, is it more valuable to give half of their stake to the other project in hopes of future collusion? If future collusion isn't a sure thing, that's probably not going to be a wise choice. Thus, changing out the group of delegates for Quadratic Voting substantially lowers the risk of this collusion.

6.2 Simulation Results

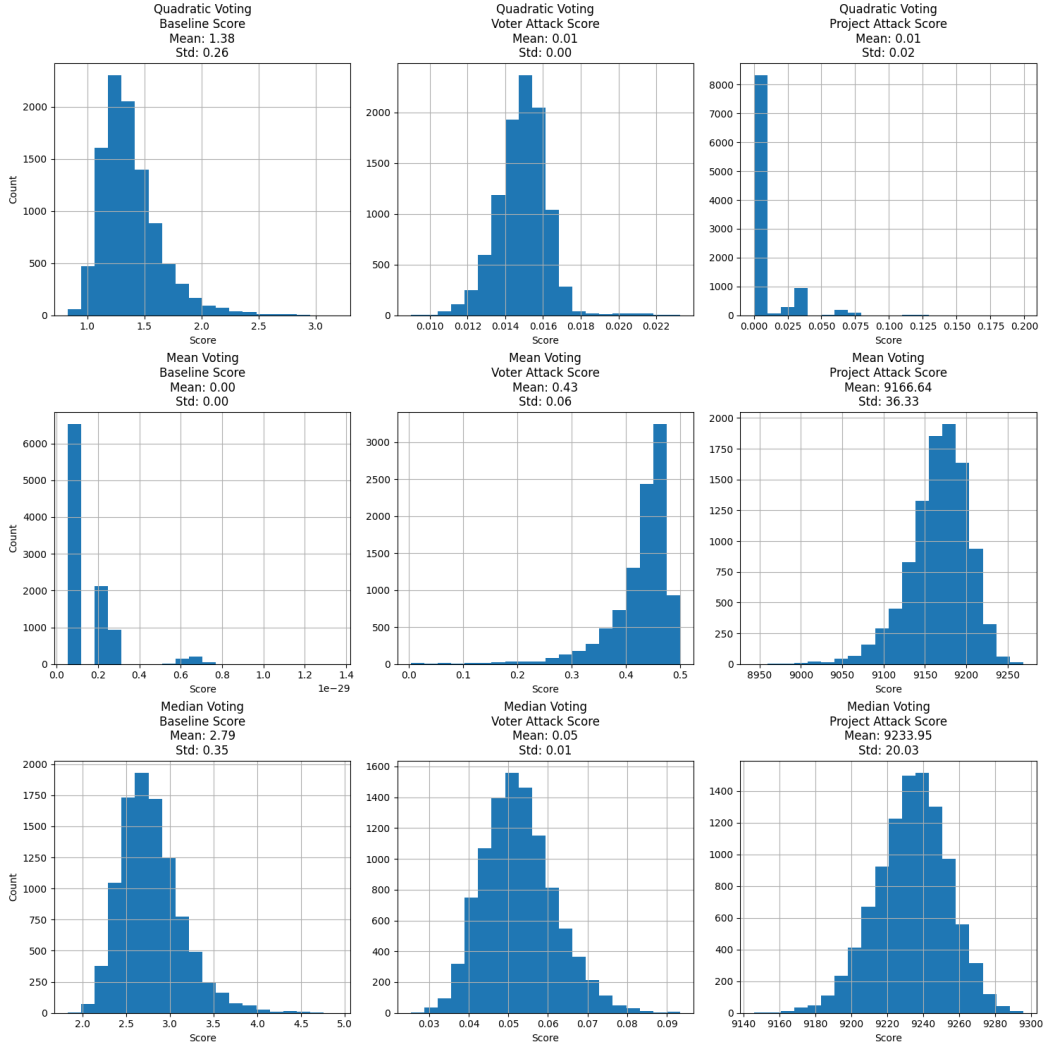


Figure 1: Distribution of manipulation scores across voting mechanisms. Each row shows a different voting mechanism (Quadratic, Mean, and Median), while columns represent Baseline, Voter Attack, and Project Attack scenarios. The x-axis shows manipulation scores and y-axis shows frequency counts across 10,000 simulation iterations.

Our simulation analysis substantiates the theoretical vulnerabilities proven in Section 4, while providing quantitative measures of each mechanism's resilience to attack. Figure 1 presents the distribution of manipulation scores across baseline conditions and attack vectors for each mechanism.

Drawing from the simulation results, we also highlight several observations:

Obversation 3: Quadratic Voting Demonstrates strong empirical resistance to manipulation

Quadratic voting demonstrates strong empirical resistance to manipulation. Under baseline conditions, the mechanism produces a normally distributed score centered at approximately 1.38 with a standard deviation of ≈ 0.26 . More notably, voter attacks yield minimal manipulation scores in the $[0.013, 0.018]$ range, indicating robust defense against individual manipulation attempts. Project-level attacks show similar resilience, albeit with some outliers > 0.1 . These represent the smallest percentage differences relative to the other voting mechanisms.

Obversation 4: Both mean and median voting are highly vulnerable to project attacks

Both mean and median voting mechanisms demonstrate severe vulnerability to project-level phantom attacks, with manipulation scores orders of magnitude larger than their baseline behaviors. Mean voting shows project attack scores concentrated in the $[9050, 9250]$ range, while median voting exhibits scores in the $[9140, 9300]$ range. These high scores align with our theoretical analysis of phantom vote attacks, where coordinated effort from multiple projects can significantly distort allocation outcomes. The normally distributed nature of these attack scores suggests that this vulnerability is systematic rather than probabilistic, indicating a fundamental weakness in both mechanisms against coordinated manipulation.

Obversation 5: Voter attacks affect mean more than median, project attacks vice versa

The differential impact of attack vectors on mean and median voting reveals an interesting trade-off. Under voter attacks, mean voting shows manipulation scores clustering around $[0.4, 0.5]$, while median voting demonstrates more modest manipulation in the $[0.04, 0.07]$ range, showing approximately an order of magnitude difference. However, this relationship inverts under project attacks, where median voting shows marginally higher vulnerability (scores centered around 9233) compared to mean voting (scores centered around 9166). This empirically validates our theoretical analysis from Observation 1, suggesting that median voting suffers far more than mean in large-scale voting attacks, as a large number of colluding voters, as in the project attack case, will create a more significant cliff between ordered results. Conversely, when it comes to attacks involving individual voters, mean is an order of magnitude more susceptible to singular outliers than the median.

6.3 Practical Recommendations

Table 2: Summary Evaluation of Voting Mechanisms

Mechanism	Evaluation
Quadratic Voting	Very robust to both voter and project attacks compared to other mechanisms. To combat potential voter collusion, could be implemented with voter rotation periods, as well as with some minimum allocation thresholds.
Mean Voting	Simplicity in implementation and voter comprehension, no deviation in voter preferences vs. the baseline. But vulnerable to both voter attacks and project attacks. Slightly more resilient to large-scale project attacks than median.
Median Voting	Increased resistance to voter attacks vs. mean voting, but shows severe vulnerability to coordinated efforts due to the discrete step function. Would need complex mechanism re-engineering to be practically feasibility, but there is a big tradeoff for voter comprehension.

Table 2 contains a summary of our practical recommendations derived from both our theoretical and simulation observations. To aid practitioners have access to our simulation framework, modify

simulation parameters, and see how voting results may vary from a single set of preferences, we have released an open-source interactive prototype dashboard to demonstrate our results ³.

Cross-mechanism comparison reveals a clear ordering in manipulation resistance. Quadratic voting provides superior protection across all attack vectors, with manipulation scores consistently an order of magnitude lower than alternative mechanisms. Median voting offers moderate protection against individual voter attacks but remains vulnerable to coordinated efforts. Mean voting demonstrates the highest overall susceptibility to manipulation, particularly to project-level coordination. While quadratic voting shows inherent resilience, this protection assumes non-repeated interactions between voters, as discussed in Section 4.3. The stark contrast in project attack scores (0.25 vs 9200+) between quadratic and other mechanisms emphasizes the practical importance of this theoretical distinction.

7 Further Considerations

Median Voting Variations For simplicity of theoretical and simulation demonstration, we modeled a standard median voting system. We note that within Round 3 and Round 4 of Optimism’s RetroPGF mechanisms, Optimism modified the median voting mechanism to have quorum caps in Round 3 and maximum and minimum thresholds in Round 4 [11] [12]. While the intent of these mechanisms may have been to mitigate Phantom Vote style attacks, we believe that voter and project attacks remain plausible threats. In a quorum cap model for example, a group of n attackers and a quorum of q would need to simply calibrate an epsilon of $\epsilon > q/n$. Moreover, we hypothesize modified median mechanisms such as the min/max cap in RetroPGF Round 4 artificially collapse voter preferences, which may widely affect voter project choices, especially under our model that assumes preferences follow a Pareto distribution due to voter time and resource constraints. The artificial caps introduced onto the median seem conceptually very similar to quadratic voting, which is far more conceptually simpler to explain to voters. A comparison of these modified hybrid mechanisms with standard models of quadratic and median voting is an area for further research.

Extending Simulation Parameters Within our simulation framework, we made several model assumptions, including using a Pareto distribution for voter preferences, constants for voter weights, and using available project and voter data to approximate a realistic voting scenario. An area of further research is in evaluating this same set of voting mechanisms on different voter and project parameters, and using different distributions to model both voter preference and voter weight, empirically validate our assumptions against real-world, non-aggregated voter preference data. Our online simulation dashboard provides an extend toolkit to experiment with all of these additional parameters and with custom voter preference datasets.

Conducting Voting Experiments Our theoretical and simulation framework also lays the theoretical groundwork for future governance experiments. With quadratic voting being our tentative recommendation as a governance mechanism, we could experiment with implementing mechanisms for rotating voter sets for different projects to reduce collusion, while also balancing potential benefits of anti-collusion against the importance of maintaining expertise knowledge. This balance may be particularly crucial in the case of retroactive funding systems where there requires more sophisticated understanding of specific verticals and projects, and there may be a limited pool of informed voters. Another extension of this framework could be towards domains of decentralized voting outside of RetroPGF, such as in Optimism’s Token House, to evaluate if similar mechanisms would be valuable there too.

³A public prototype deployment with multiple customizable parameters can be found here: <https://opt-voting.vercel.app>

References

- [1] Kenneth J. Arrow. *Social Choice and Individual Values*. Yale University Press, New Haven, 1951.
- [2] Optimism Atlas. RPGF Recipients. <https://atlas.optimism.io/round/results>. Accessed on: 2025-01-31.
- [3] James Austgen, Andrés Fábrega, Sarah Allen, Kushal Babel, Mahimna Kelkar, and Ari Juels. DAO Decentralization: Voting-Bloc Entropy, Bribery, and Dark DAOs. *arXiv preprint*, 2023. <https://arxiv.org/abs/2311.03530>.
- [4] Duncan Black. On the rationale of group decision-making. *Journal of Political Economy*, 56(1):23–34, 1948.
- [5] Jean-Antoine-Nicolas de Caritat Condorcet, Iain McLean, and Fiona Hewitt. *Condorcet: foundations of social choice and political theory*. E. Elgar, Aldershot, Hants, England ; Brookfield, Vt, 1994.
- [6] DeepDAO. DeepDAO Organizations. <https://deepdao.io/organizations>. Accessed 2024-11-02.
- [7] Compound Docs. Governance. <https://docs.compound.finance/v2/governance/>. Accessed 2024-10-22.
- [8] Optimism Citizen House Docs. How retro funding works. <https://community.optimism.io/citizens-house/how-retro-funding-works>. Accessed 2024-11-02.
- [9] Francis Galton. Vox populi. *Nature*, 75:450–451, 1907.
- [10] Allan Gibbard. Manipulation of voting schemes: A general result. *Econometrica*, 41(4):587–601, 1973.
- [11] Optimism Governance. Optimism Retro Funding Round 3. <https://community.optimism.io/citizens-house/rounds/retropgf-3>. Accessed on: 2025-02-03.
- [12] Optimism Governance. Optimism Retro Funding Round 4. <https://community.optimism.io/citizens-house/rounds/retropgf-4>. Accessed on: 2025-02-03.
- [13] Andrew B. Hall and Eliza R. Oak. What Kinds of Incentives Encourage Participation in Democracy? Evidence from a Massive Online Governance Experiment. 2023. https://andrewbenjaminhall.com/Hall_Oak_Airdrop_Effects_on_Participation.pdf.
- [14] Samer Hassan and Primavera De Filippi. Decentralized autonomous organization. *Internet Policy Review*, 10(2):1–10, 2021.
- [15] Steven Lalley and E. Glen Weyl. Quadratic voting: How mechanism design can radicalize democracy. *American Economic Association Papers and Proceedings*, 1(1):1–5, 2018.
- [16] Christian List. Social Choice Theory. In Edward N. Zalta and Uri Nodelman, editors, *The Stanford Encyclopedia of Philosophy*. Metaphysics Research Lab, Stanford University, Winter 2022 edition, 2022. <https://plato.stanford.edu/archives/win2022/entries/social-choice/>.
- [17] Eric A. Posner and E. Glen Weyl. Quadratic voting and the public good, 2017. Accessed on: 2024-12-05.
- [18] Tanusree Sharma, Yujin Kwon, Kornrapat Pongmala, Henry Wang, Andrew Miller, Dawn Song, and Yang Wang. Unpacking how decentralized autonomous organizations (daos) work in practice. *ArXiv Preprint*, 2023. <https://arxiv.org/abs/2304.09822>.
- [19] James Surowiecki. *The Wisdom of Crowds*. Knopf Doubleday Publishing Group, Westminster, 2005.